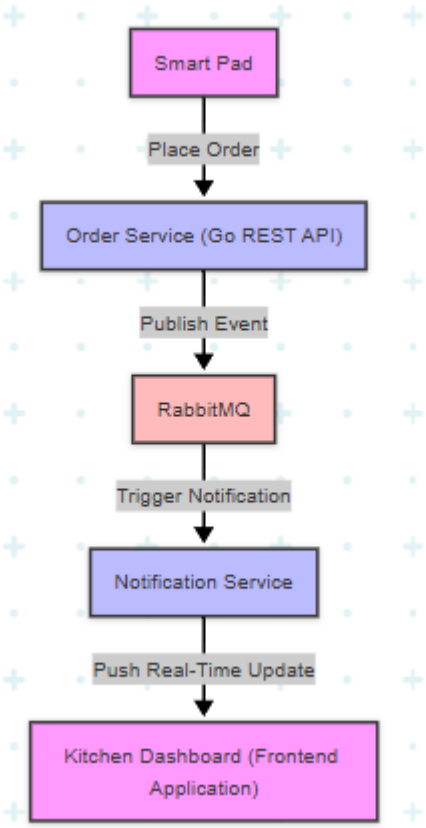
Bistro 92 Restaurant Management System

A) Quick Fixes

Q1: Three Essential Features for Customer Satisfaction and Order Processing (25 points)

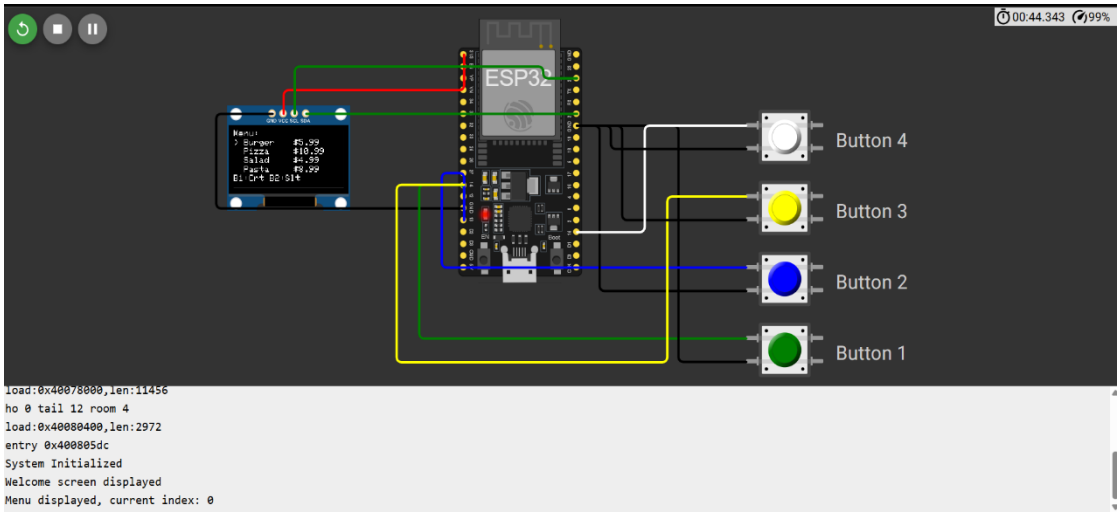
1. **Real-Time Ordering and Status Updates**
   - Customers and kitchen receive instant updates when an order is placed. (WebSocket-based notification service)
2. **Easy Menu Browsing and Item Selection**
   - Easy-to-navigate smart pad interface allowing users to browse, select, and adjust orders.
3. **Customer Feedback (Future Implementation)**
   - Data collection regarding customer experience and satisfaction level as per rating.

Diagram:

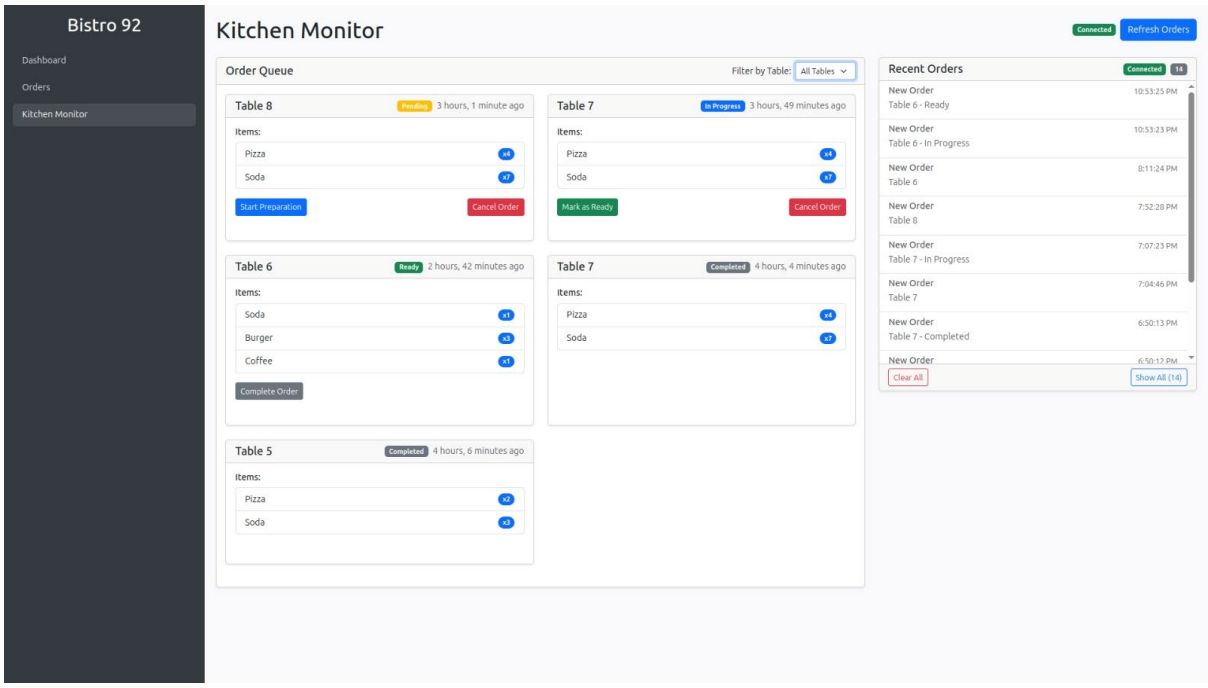


Q2: Two Design Principles for Smart Pad Interface (25 points)

1. **Minimalist User Interface for gathering Input**



2. **Monitoring System by the backend User:**



Q3: Three Security Vulnerabilities and Solutions (30 points)

Vulnerability	Problem	Solution
Unauthorized Access	Random people accessing the system.	Each table ESP IP is assigned a unique Static ID under the Restaurant network using DHCP server
Order Tampering	Hackers messing with order data	Communications are secured using HTTPS and secure WebSocket (WSS).
Device Theft (Future Improvement)	Physical theft of smart pads.	Implement device locking and disable features upon network disconnection.

Q4: Two Strategies to Keep System Responsive During Peak Hours (30 points)

1. **Horizontal Scaling with Load Balancer (Future Improvement)**
    - Using Nginx/HAProxy to distribute traffic evenly across multiple instances of services (Order Service, Notification Service).
2. **Message Queuing with RabbitMQ**
    - Orders placed are queued, ensuring backend services can process them during overload.

Q5: Method to Integrate Existing Inventory System (40 points)

Integration Method:

- Created a lightweight **Inventory Sync Microservice** that listens for new orders.
- After order placement, this service updates stock levels by reducing ingredients used per dish and showing it to real time dashboard.
- Use RabbitMQ events to trigger inventory updates.

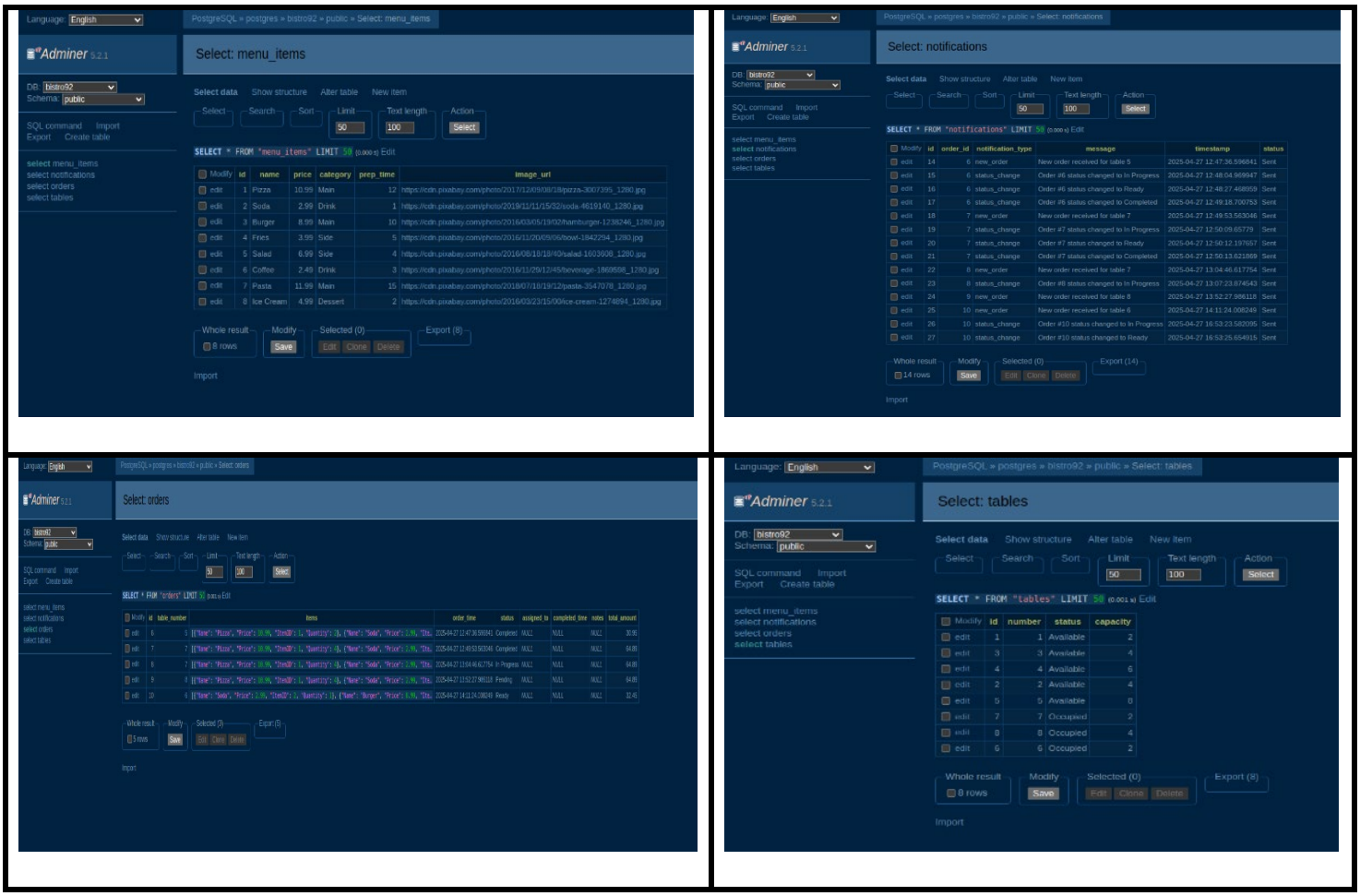
Steps:

1. Order placed -> Event published to RabbitMQ.
2. Inventory Sync Service consumes event.
3. Reduces inventory count in existing database.

This step is described in more details clicking the following Drive link for the video: https://drive.google.com/drive/folders/1W2xmc-nbk3S5A7ucsBNhbN7sBHO45blx?usp=sharing

B) Tech Tricks

Q1 (70 points): Design a database schema for Bistro 92 to track users, orders, menu items, tables, and payments, optimized for fast queries.

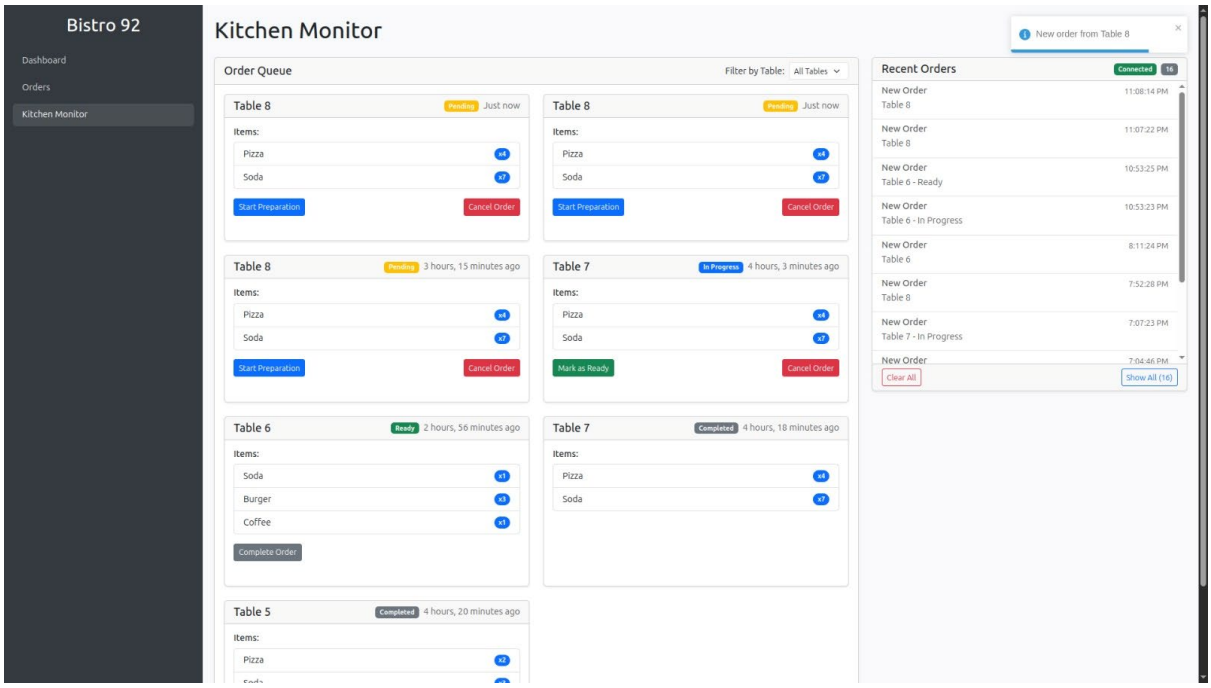


Q2. Q2 (80 points): Write an SQL query to retrieve all orders from the last hour, including table number, items ordered, and order time, optimized for speed.

```
SELECT
    id,
    table_number,
    items,
    order_time,
```

```
    status
FROM
    orders
WHERE
    order_time >= NOW() - INTERVAL '1 hour'
ORDER BY
    order_time DESC;
```

Q3 (90 points): Implement a feature to notify kitchen staff in real-time when a new order is placed, and describe your tech stack.



✅ **Tech Stack:**

- **Backend:** Go (Gin framework) — receives orders.
- **Messaging:** RabbitMQ — passes new order event.
- **WebSocket Server:** Node.js/Go — keeps live connections open to Kitchen UI.
- **Frontend:** React App — listens for WebSocket push, auto-updates kitchen screen.

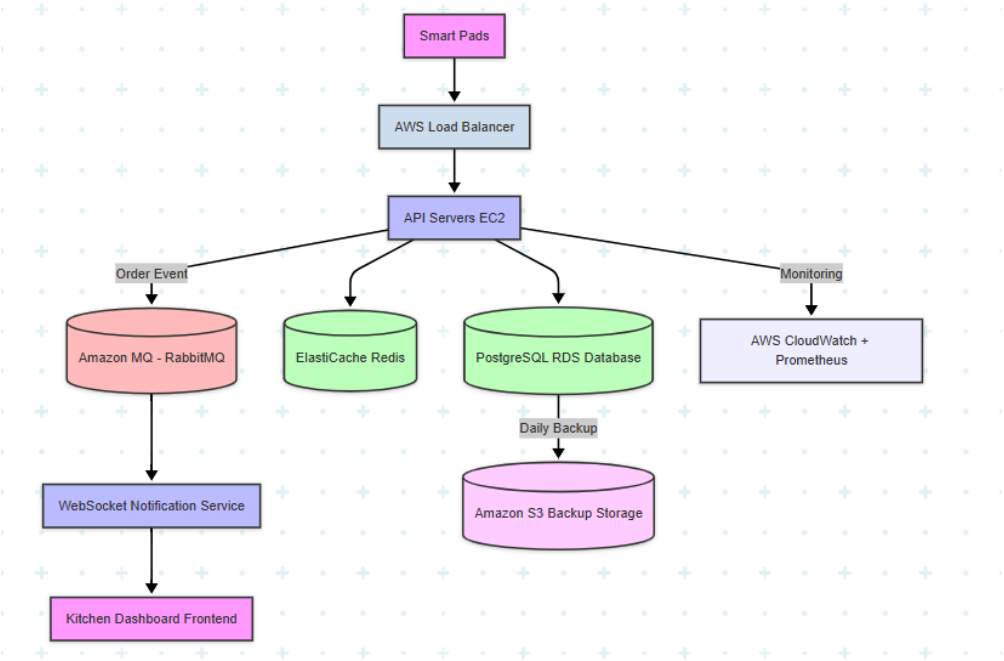
Flow:

Smart Pad → Backend API → RabbitMQ → WebSocket Notification → Kitchen Dashboard

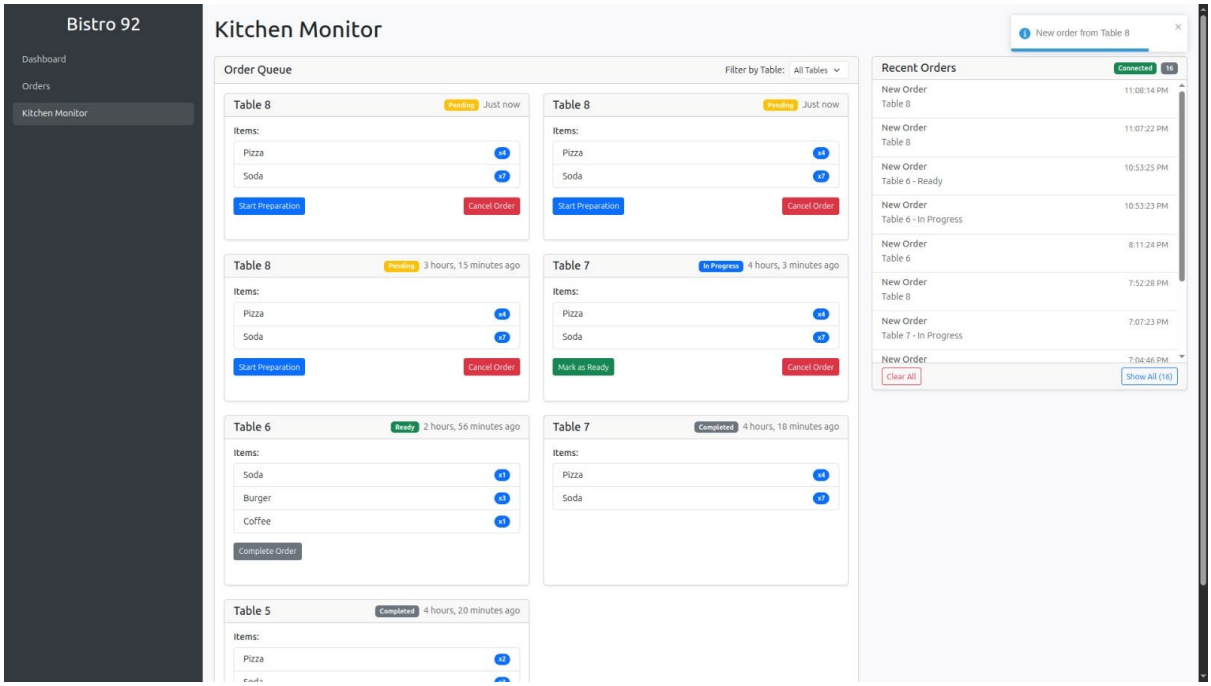
✅ **Benefit:**
Kitchen staff sees a new order pop-up instantly without refreshing

Q4 (100 points): Describe a cloud-based system architecture for real-time updates, data storage, and smart pad communication, ensuring low latency and high availability.

Smart Pads send orders through an AWS Load Balancer to API Servers, which save data in PostgreSQL (RDS) and cache it in Redis for speed.
Orders trigger real-time updates via RabbitMQ and WebSocket to the kitchen dashboard.
Backups happen daily to S3, and everything is monitored by Prometheus and CloudWatch for instant recovery.



Q5 (110 points): Design a real-time dashboard showing pending orders, average fulfillment time, and total sales, specifying tools and justifying your choices.



C) Bonus Boosters

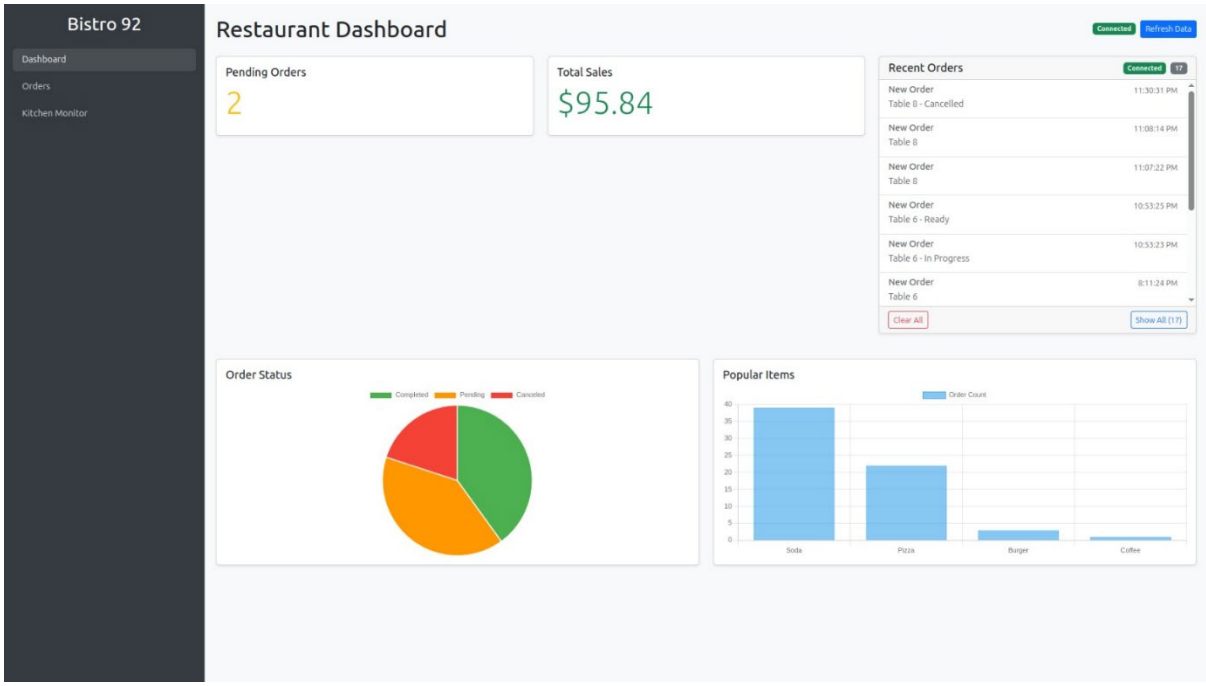
Q1 (250 points): Design a RESTful API for order placement that handles high concurrency without errors, detailing endpoints, request/response formats, and concurrency controls

It can be observed from the following GitHub link:
https://github.com/Ridwan414/bistro-92

Q2 (250 points): Enhance your API to support extreme scalability, processing numerous simultaneous orders without data loss, and explain your strategies.

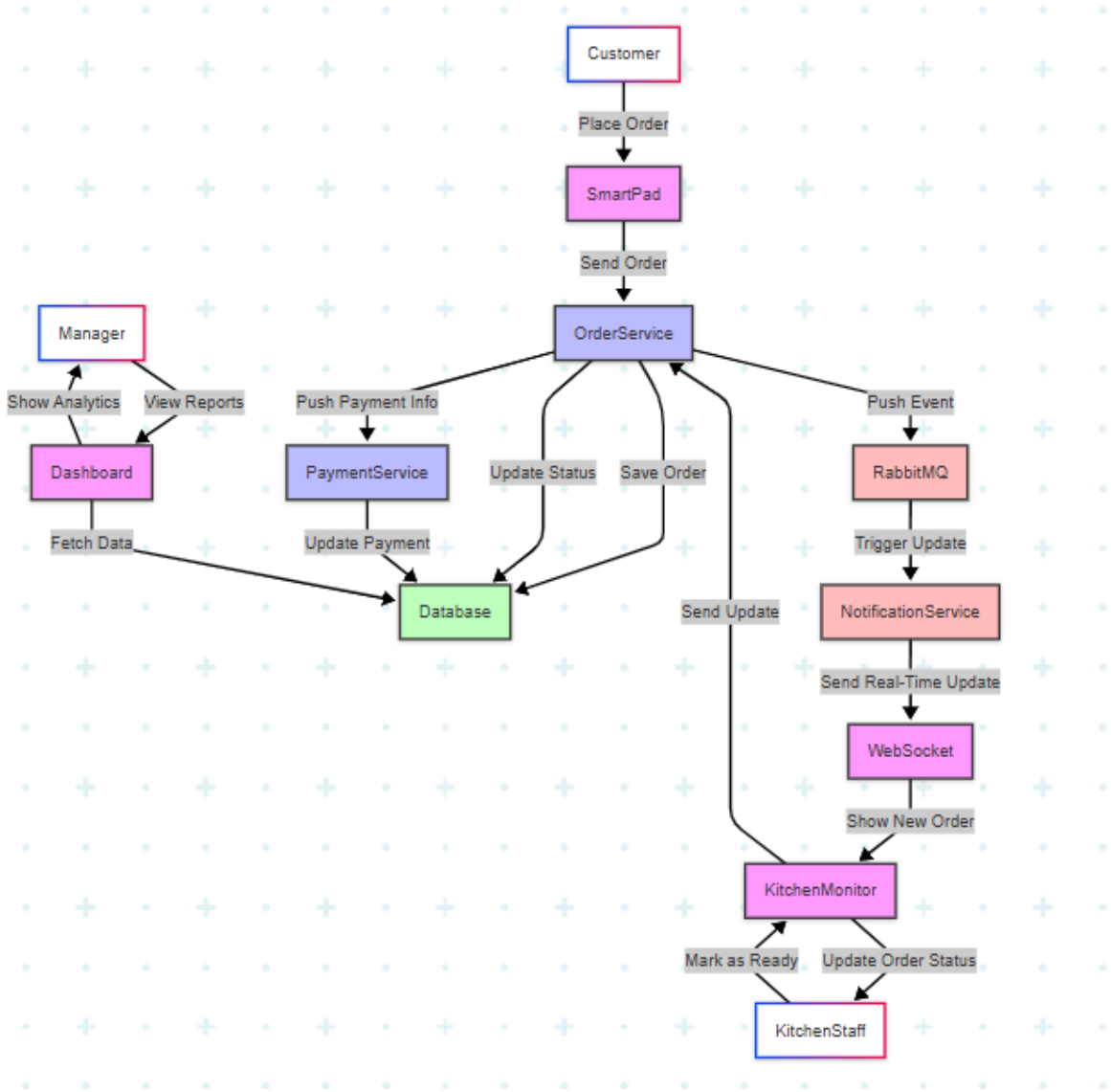
Used temporal workflow engine where queue-based workflow added And the APIs has been created using Golang's go routine. We have maintained micro service architecture that can be scalable later using k8s . All the service has docker file.

D) Big Idea (up to 300 Points)



1. The Database created form the user input servers as the big resources for the future Capital investment regarding the Restaurant and it's expandability
2. Querying the Database to track the user's preferability over the Set Menu's and highest order placed record
3. We also implemented a feature so that the Chef / authorized person form the kitchen monitoring can give a approximate time for the order to be cooked which wiil be shown in the Smart-Pad reducing the Broadness of the customer

The Overall System Architecture:



Conclusion

We made sure Bistro 92 runs smoothly, even on the busiest nights! Orders go straight from the table to the kitchen in real-time, and everything — from inventory to payment to analytics — updates automatically behind the scenes.